

APUNTES - CURSO DE ARQUITECTURAS CSS

Guía de estudio clara y práctica

1. Fundamentos de arquitectura CSS

¿Qué es una arquitectura CSS?

Una arquitectura CSS es una forma organizada de escribir y estructurar los estilos de un proyecto. La idea es que el CSS sea fácil de entender, reutilizar, mantener y ampliar cuando el proyecto crezca.

Un CSS bien organizado busca evitar reglas repetidas, conflictos de estilos y selectores difíciles de mantener.

Principios importantes

- SOLID: ayuda a pensar en responsabilidades claras y código más mantenible.
- DRY (Don't Repeat Yourself): evitar repetir la misma lógica o estilos innecesariamente.
- Inmutabilidad: procurar que una regla o componente no se modifique accidentalmente desde diferentes lugares.

Lo importante es entender que una arquitectura no es solamente ordenar carpetas: también es definir reglas para escribir CSS de manera consistente.

Lo que debo recordar

- La arquitectura CSS sirve para mantener el código organizado cuando el proyecto crece.
- El objetivo es conseguir CSS limpio, reutilizable, mantenible y escalable.

2. BEM — Block, Element, Modifier

BEM es una metodología para nombrar y organizar clases CSS de una manera predecible. Sus tres partes son Block (bloque), Element (elemento) y Modifier (modificador).

Bloque

Es un componente independiente de la interfaz.

`<article class="card">...</article>`

```
- <header>
-   <div class="logo_name">
-       
-       </div>
-   <p class="name">Ralerv</p>
-   <nav class="nav-header">
-       <span class="menu-icon--span img">
-           
-       </span>
-       <ul class="nav-list" id="nav-list">
-           <li><a class="nav-link" href="#about">Acerca de mí</a></li>
-           <li><a class="nav-link" href="#projects">Proyectos</a></li>
-           <li><a class="nav-link" href="#social-tree">Encuentrame</a></li>
-           <li><a class="nav-link" href="#extras">Extras</a></li>
-       </ul>
-   </nav>
- </header>
```

Elemento

Es una parte del bloque que depende de él.

`<h2 class="card__title">Título</h2>`

```
+ <nav class="navbar">
+   
+   <p class="navbar__name">Ralerv</p>
+   
+   <ul id="nav-list" class="navbar__list">
+     <li class="navbar__link"><a href="#about">Acerca de mí</a></li>
+     <li class="navbar__link"><a href="#projects">Proyectos</a></li>
+     <li class="navbar__link"><a href="#social-tree">Encuentrame</a></li>
+     <li class="navbar__link navbar__link--extras"><a href="#extras">Extras</a></li>
+   </ul>
+ </nav>
```

Modificador

Representa una variación del bloque o de un elemento.

`<article class="card card--featured">...</article>`

```
<template>
- <footer class="wrapper">
-   <div class="footer-container">
-     <div class="footer-container__social-container">
-       <div
+ <div class="wrapper">
+   <footer class="footer">
+     <ul class="social-media">
+       <li
+         v-for="(item, index) in networks"
+         :key="index"
+         class="social-network"
+       >
+         class="social-media__list-item">
+           <a href="item.link" target="_blank">{{ item.name }}</a>
+         </div>
+       </li>
+     </ul>
+     <div class="copyright">
+       <p class="copyright__location">Guayaquil, Ecuador</p>
+       <p class="copyright__signature">2022, DEVELOPED BY LUIS REYES</p>
+     </div>
-     <div class="footer-container__copyright-container">
-       <p class="location">Guayaquil, Ecuador</p>
-       <p class="signature">2022, DEVELOPED BY LUIS REYES</p>
-     </div>
-   </div>
- </footer>
+ </footer>
+ </div>
</template>
```

Errores comunes

- Crear nombres demasiado complicados o difíciles de leer.

- Usar BEM de forma innecesariamente profunda.
- Confundir un elemento con un bloque independiente.
- Depender de selectores HTML o ID en lugar de clases.

Refactorización con BEM

Refactorizar significa reorganizar un código que ya funciona para hacerlo más claro y mantenible. En las clases prácticas del curso se aplica BEM sobre proyectos reales y también sobre componentes de Vue y React.

BEM en Vue y React

La idea principal se mantiene: los componentes pueden tener nombres de clases que expresen claramente su bloque, sus elementos y sus variaciones. En React también se muestra su uso junto con Styled Components.

Lo que debo recordar

- BEM se enfoca principalmente en una forma clara de nombrar clases.
- No es necesario hacer nombres enormes: debo buscar claridad.
- BEM ayuda a saber qué pertenece a qué componente.

3. Atomic Design

Atomic Design organiza una interfaz desde piezas pequeñas hasta estructuras completas. La idea es construir componentes reutilizables y después combinarlos.

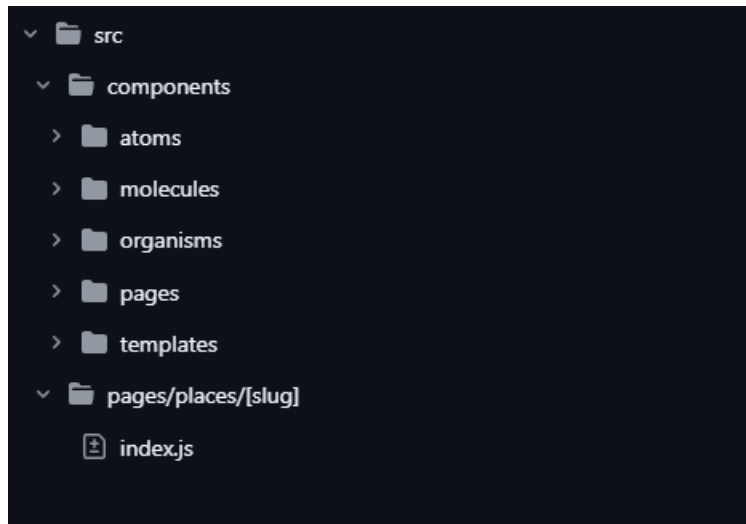
Los cinco niveles

- Átomos: piezas pequeñas, como botones, títulos o enlaces.
- Moléculas: combinación de varios átomos.
- Organismos: combinación de moléculas y otros componentes para formar una sección.
- Plantillas: estructura general de una página.
- Páginas: una plantilla con contenido real.

Ejemplo sencillo: un botón puede ser un átomo; un campo de búsqueda con icono y botón puede formar una molécula; una barra de navegación puede ser un organismo.

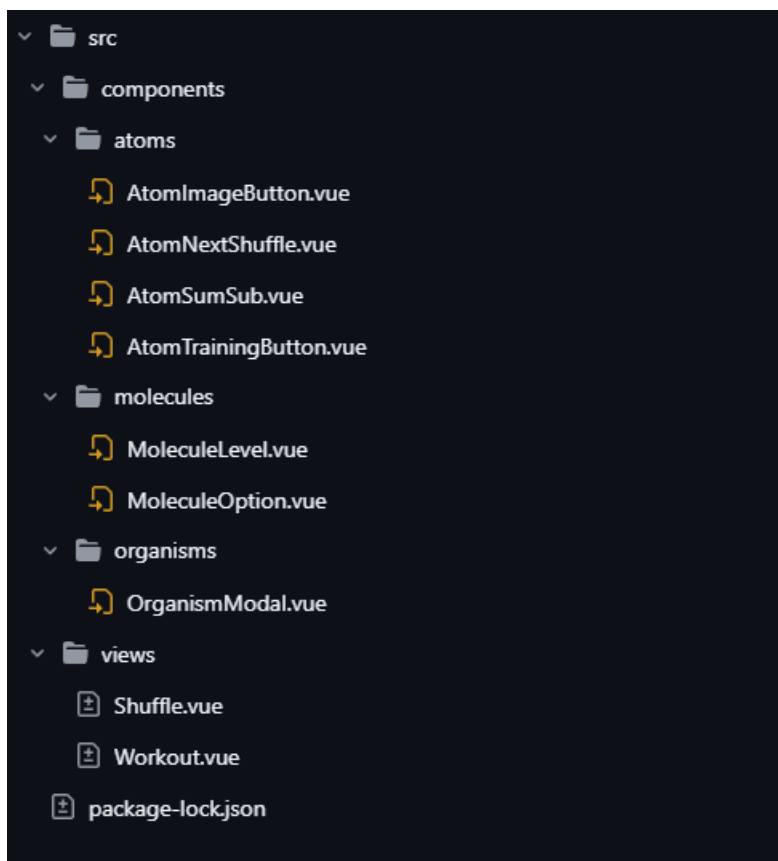
Atomic Design en React y Vue

En los proyectos del curso se aplica la idea de organizar componentes según estos niveles. En Vue también se relaciona con la organización de carpetas y componentes.



Lo que debo recordar

Atomic Design me ayuda a pensar la interfaz como piezas que se combinan, desde lo más pequeño hasta una página completa.



4. OOCSS — Object Oriented CSS

OOCSS significa Object Oriented CSS. Su objetivo es favorecer la reutilización del CSS mediante objetos o estructuras reutilizables.

<pre> - \$normal: #a8a878; - \$fire: #f08030; - \$water: #6890f0; - \$grass: #78c850; - \$electric: #f8d030; - \$ice: #98d8d8; - \$fighting: #bf2f27; - \$poison: #a040a0; - \$ground: #e0c068; - \$flying: #a890f0; - \$psychic: #f85888; - \$bug: #a8b820; - \$rock: #b8a038; - \$ghost: #705898; - \$dark: #705848; - \$dragon: #7038f8; - \$steel: #b8b8d0; - \$fairy: #f0b5bb; </pre>	<pre> + \$colors: (+ normal: #a8a878, + fire: #f08030, + water: #6890f0, + grass: #78c850, + electric: #f8d030, + ice: #98d8d8, + fighting: #bf2f27, + poison: #a040a0, + ground: #e0c068, + flying: #a890f0, + psychic: #f85888, + bug: #a8b820, + rock: #b8a038, + ghost: #705898, + dark: #705848, + dragon: #7038f8, + steel: #b8b8d0, + fairy: #f0b5bb +); </pre>
--	--

Principio 1: separar estructura y apariencia

La estructura define cómo se organiza un componente, mientras que la apariencia o 'skin' define aspectos visuales como colores o estilos.

```

@@ -1,8 +1,17 @@
- <section class="pokemonCard" *ngIf="pokemonName">
+ <section
+   *ngIf="pokemonName"
+   class="pokemon-card pokemon-card-size">
  <h3>{{ pokemonName }}</h3>
+ <img [src]="pokemonSprite" [alt]="pokemonName" *ngIf="pokemonSprite">
- <ul *ngIf="pokemonTypes.length > 0">
-   <li *ngFor="let type of pokemonTypes" [className]="type">
+ <img
+   *ngIf="pokemonSprite"
+   [src]="pokemonSprite"
+   [alt]="pokemonName">
+ <ul
+   *ngIf="pokemonTypes.length > 0">
+   <li
+   *ngFor="let type of pokemonTypes"
+   [class]="type"
+   class="pokemon-tag">

```

Principio 2: separar contenedor y contenido

La idea es evitar que un estilo dependa demasiado de estar dentro de un contenedor específico. Así los objetos pueden reutilizarse en diferentes partes.

```
.button { padding: 10px 16px; }
```

```
.button--primary { background: blue; }
```

Aquí la estructura básica del botón puede reutilizarse y la apariencia puede cambiar.

Refactorización con OOCSS y SASS

La práctica del curso lleva estos principios a un proyecto Angular usando SASS. La meta de la refactorización es detectar estilos repetidos y convertirlos en estructuras reutilizables.

Lo que debo recordar

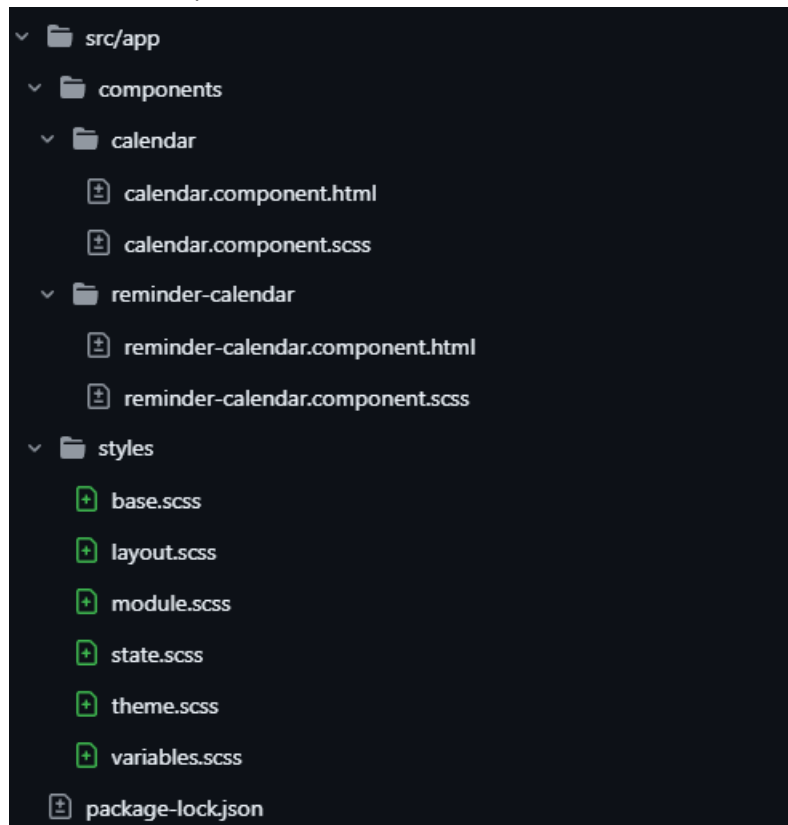
OOCSS busca principalmente reutilización y separación de responsabilidades entre estructura y apariencia.

5. SMACSS

SMACSS significa Scalable and Modular Architecture for CSS. Su idea principal es categorizar los estilos para saber dónde encaja cada regla.

Categorías

- Base: estilos generales y elementos básicos, como títulos, enlaces y botones.
- Layout: estructura principal de la página y distribución.
- Modules: componentes o módulos reutilizables.
- State: estados de los elementos, por ejemplo activo, oculto o seleccionado.
- Theme: estilos relacionados con la apariencia o tema visual.



La ventaja de categorizar es que el proyecto se vuelve más fácil de entender y mantener. En la práctica del curso se aplica la organización a un proyecto Angular usando SASS.

Lo que debo recordar

SMACSS se centra en clasificar y organizar los estilos según la función que cumplen.

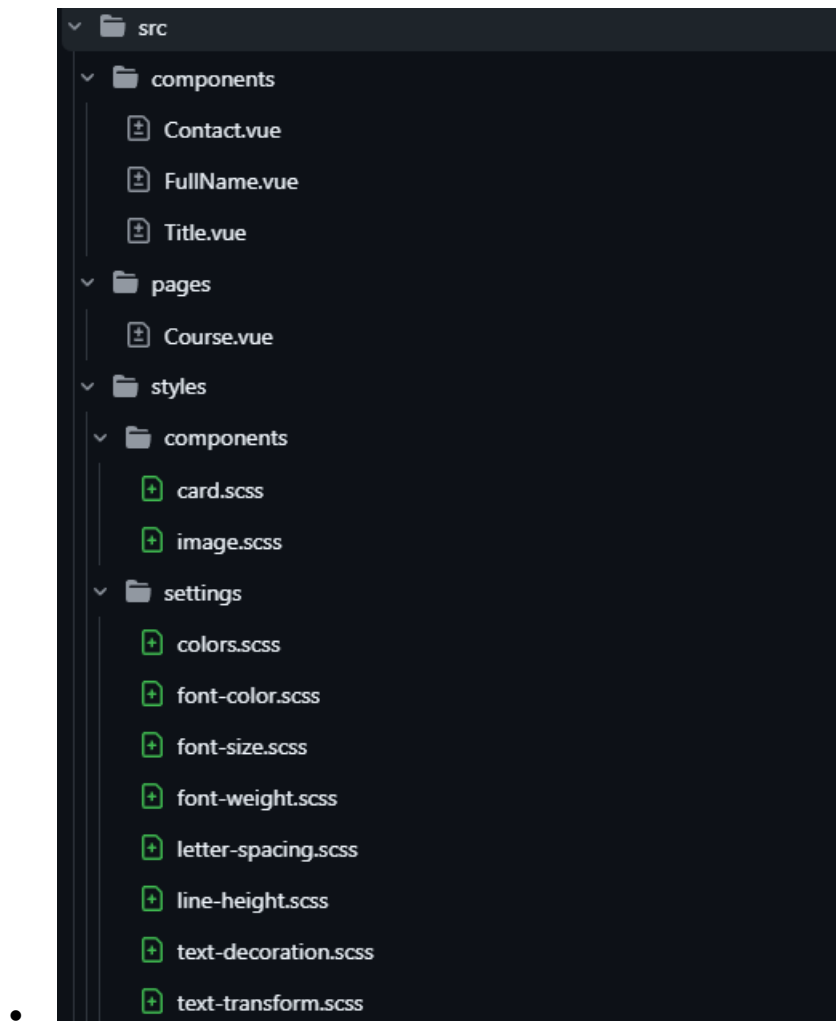
```
<div class="fieldReminder">
  <label class="resaltedTitle">Reminder</label>
<div class="field-reminder">
  <label class="title">Reminder</label>
  <input type="text" value="Remember me" />
</div>
```

6. ITCSS — Inverted Triangle CSS

ITCSS significa Inverted Triangle CSS. Organiza el CSS en capas que van desde reglas más generales hasta reglas más específicas. El concepto del triángulo invertido ayuda a controlar la especificidad y la cascada.

Idea de las capas

- Settings: configuraciones y variables.
- Tools: herramientas, funciones o mixins.
- Generic: estilos generales y normalizaciones.
- Elements: estilos de elementos HTML.
- Objects: estructuras reutilizables.
- Components: componentes concretos de la interfaz.
- Utilities: clases pequeñas y específicas que aplican una utilidad.



El orden puede variar según el proyecto. Lo importante es que las capas avanza de lo más general a lo más específico, evitando que todo el CSS tenga la misma fuerza.

Limitaciones en proyectos pequeños

ITCSS puede resultar demasiado estructurado para un proyecto pequeño. La arquitectura debe adaptarse al contexto: no siempre tiene sentido aplicar toda la estructura posible si el proyecto es sencillo.

Lo que debo recordar

ITCSS ayuda a controlar la cascada y la especificidad organizando los estilos por capas.

7. Tailwind CSS

Tailwind CSS es un framework que utiliza clases de utilidad para construir interfaces directamente en el HTML o en los componentes.

```
<button class="px-4 py-2 rounded">Guardar</button>
```

En lugar de crear una clase CSS para cada componente desde cero, se combinan utilidades que representan propiedades concretas.

- Permite trabajar rápidamente con clases de utilidad.

- Ayuda a mantener decisiones visuales consistentes cuando se configura correctamente.
- Puede utilizarse en proyectos modernos junto con frameworks de frontend.

Arquitectura profesional con Tailwind

La clase del curso presenta Tailwind como una alternativa para organizar estilos y después plantea cómo elegir entre BEM, ITCSS y frameworks según las necesidades reales del proyecto.

8. ¿Cómo elegir una arquitectura CSS?

No existe una metodología que sea siempre la mejor. La decisión depende del tamaño del proyecto, la cantidad de personas que trabajan en él, el nivel de reutilización necesario y las herramientas utilizadas.

- Proyecto pequeño → una arquitectura muy compleja puede añadir trabajo innecesario.
- Proyecto grande → una metodología clara puede ayudar mucho a mantener el CSS.
- Equipo grande → las reglas de organización ayudan a que todos escriban CSS de manera consistente.
- Framework de utilidades → puede ser conveniente usar Tailwind si encaja con el proyecto.

En un proyecto real también es posible combinar ideas de distintas metodologías si eso resuelve mejor las necesidades del equipo.

9. Comparación rápida

Metodología	Idea principal	Me ayuda a...
BEM	Bloque, elemento y modificador	Nombrar clases de forma clara
Atomic Design	Átomos → páginas	Organizar componentes reutilizables
OOCSS	Separar estructura y apariencia	Reutilizar estilos
SMACSS	Categorizar estilos	Organizar el CSS por función
ITCSS	Capas de menor a mayor especificidad	Controlar cascada y especificidad
Tailwind	Clases de utilidad	Construir interfaces rápidamente